ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

ассистент		Д.А. Кочин
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №6

НЕПРЕРЫВНАЯ ИНТЕГРАЦИЯ В ОБЛАКЕ С ИСПОЛЬЗОВАНИЕМ GITHUB ACTIONS

по курсу: ОПЕРАЦИОННЫЕ СИСТЕМЫ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №	4236		А.В. Панченко
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2024

Цель работы:

Изучение принципов организации непрерывной интеграции приложений с использованием облачных технологий.

Задание на лабораторную работу:

1. Добавить в репозиторий предыдущей лабораторной работы файл конфигурации .github/workflows/tests.yml, управляющий созданием виртуальной машины в облаке, на которой автоматически будет запускаться сборка и тестирование проекта. Примеры оформления файла конфигурации можно найти в репозиториях лабораторных работ №№1-5, в официальной документации, а также с помощью поиска в Интернете. Задачу (job) в файле конфигурации необходимо назвать test, использование других имен приведет к потере баллов. Команды в разделе steps задачи должны запускать сборку той части предыдущей лабораторной работы, которая в соответствии с вариантом была выполнена в ОС Linux (или Mac OS). Например, если по варианту задания на предыдущую лабораторную работу необходимо было разработать серверную часть приложения под ОС Linux, а клиентскую часть приложения под ОС Windows, то в данной лабораторной работе необходимо осуществить автоматическую сборку серверной части приложения. Сборка должна осуществляться упешно (без ошибок).
2. По аналогии с предыдущей работой процесс выполнения задания должен быть разбит на задачи, которые должны быть описаны в разделе Issues репозитория. Решению каждой задачи должен соответствовать как минимум один коммит. Название задач должно начинаться с префикса Lab6:, такой же префикс должен быть в названии коммита (-ов). Коммиты и задачи без префикса засчитаны не будут.
3. Подготовить отчет о выполнении лабораторной работы и загрузить его под именем report-lab6.pdf в репозиторий. В случае использования системы компьютерной верстки LaTeX также загрузить исходный файл report-lab6.tex.
4. (**NB**! *Дополнительное задание*) Разработать как минимум один тест для

проверки правильности функционирования приложения, автоматическая сборка которого была обеспечена на предыдущем шаге. Добавить сборку и запуск теста (-ов) в файл конфигурации tests.yml.*

№ варианта	Индивидуальное задание	Протокол	Сервер	Клиент
18	13	UDP	Linux	Windows

Описание структуры конфигурационного файла:

Конфигурационный файл для CI (непрерывной интеграции) проекта на C++ написан в формате YAML и предназначен для использования в системах CI/CD, таких как GitHub Actions. Он начинается с указания названия рабочего процесса, которое обозначает его цель — "CI for Project". Далее в разделе событий определяется, что рабочий процесс будет запускаться при каждом push или pull_request в ветку master.В разделе заданий описывается одно задание с именем "test", которое будет выполняться на последней версии macOS. Внутри задания перечислены шаги, которые будут выполняться последовательно. Первый шаг — извлечение кода из репозитория с использованием действия actions/checkout@v2. Затем происходит установка окружения для C++, где обновляется Homebrew и устанавливается компилятор GCC.После этого осуществляется сборка серверной части проекта с помощью компилятора GCC и стандарта C++11. В заключительном шаге компилируются тесты, и они запускаются для проверки работоспособности кода. Таким образом, данный конфигурационный файл автоматизирует процесс разработки, обеспечивая регулярное тестирование и повышение качества кода.

Содержимое написанного конфигурационного файла:

```
name: CI for Project


on:
  push:
    branches:
      - master
  pull_request:
    branches:
      - master


jobs:
  test:
    runs-on: macos-latest  # Указываем, что рабочий процесс будет выполняться на macOS


    steps:
      - name: Checkout code
        uses: actions/checkout@v2


      - name: Set up C++ environment
        run: |
          brew update
          brew install gcc  # Установка компилятора GCC через Homebrew
```

```
- name: Build the server
  run: |
    g++ -std=c++11 -o server server.cpp  # Компиляция серверной части с указанием стандарта


- name: Run tests
  run: |
    g++ -std=c++11 -o run_tests test.cpp  # Компиляция тестов с указанием стандарта
    ./run_tests  # Запуск тестов
```

Логи сборки проекта в облаке:

Summary

Jobs

test

Run details

Usage

Workflow file

Annotations

2 warnings

test

succeeded 17 minutes ago in 36s

Search logs

Set up job 1s

```
1  Current runner version: '2.320.0'
2  ▶ Operating System
6  ▶ Runner Image
11 ▶ Runner Image Provisioner
13 ▶ GITHUB_TOKEN Permissions
17 Secret source: Actions
18 Prepare workflow directory
19 Prepare all required actions
20 Getting action download info
21 Download action repository 'actions/checkout@v2' (SHA:ee0669bd1cc54295c223e0bb666b733df41de1c5)
22 Complete job name: test
```

Checkout code 2s

```
1   ▶ Run actions/checkout@v2
12  Syncing repository: suai-os-2024f/os-task5-Oblachko-Js
13  ▶ Getting Git version info
17  Copying '/Users/runner/.gitconfig' to '/Users/runner/work/_temp/8a5022b8-ab2d-493c-9d37-5b6c8fba0fc7/.gitconfig'
18  Temporarily overriding HOME='/Users/runner/work/_temp/8a5022b8-ab2d-493c-9d37-5b6c8fba0fc7' before making global git config changes
19  Adding repository directory to the temporary git global config as a safe directory
20  /opt/homebrew/bin/git config --global --add safe.directory /Users/runner/work/os-task5-Oblachko-Js/os-task5-Oblachko-Js
21  Deleting the contents of '/Users/runner/work/os-task5-Oblachko-Js/os-task5-Oblachko-Js'
22  ▶ Initializing the repository
36  ▶ Disabling automatic garbage collection
38  ▶ Setting up auth
44  ▶ Fetching the repository
114 ▶ Determining the checkout info
115 ▶ Checking out the ref
119 /opt/homebrew/bin/git log -1 --format='%H'
120 '4b8b1f13163b293a1a793873e86e488384c5813f'
```

Summary

Jobs

test

Run details

Usage

Workflow file

test

succeeded 18 minutes ago in 36s

Search logs

Set up C++ environment 27s

```
1  ▶ Run brew update
5  ==> Updating Homebrew...
6  ==> Downloading https://ghcr.io/v2/homebrew/portable-ruby/portable-
   ruby/blobs/sha256:303bed4c7fc431a685db3c3c151d873740114adbdccd23762ea2d1e39ea78f47
7  #=#=#
8  ##O#-#
9
10 ##                                                                    4.1%
11 ######################################                              55.3%
12 ################################################################### 100.0%
13 ==> Pouring portable-ruby-3.3.6.arm64_big_sur.bottle.tar.gz
14 ==> Homebrew's analytics have entirely moved to our InfluxDB instance in the EU.
15 We gather less data than before and have destroyed all Google Analytics data:
16   https://docs.brew.sh/Analytics
17 Please reconsider re-enabling analytics to help our volunteer maintainers with:
18   brew analytics on
19 ==> Homebrew is run entirely by unpaid volunteers. Please consider donating:
20   https://github.com/Homebrew/brew#donations
21
22 Updated 6 taps (hashicorp/tap, azure/bicep, homebrew/bundle, aws/tap, homebrew/core and homebrew/cask).
23 ==> New Formulae
24 action-docs
25 aws/tap/eks-anywhere@0.20
26 azqr
27 boring
28 carapace
29 comtrya
30 decasify
31 dipc
32 fcft
33 foot
34 icu4c@74
35 icu4c@76
36 kanata
37 langgraph-cli
```

Summary

Jobs

test

Run details

Usage

Workflow file

test

succeeded 18 minutes ago in 36s

Search logs

Set up C++ environment 27s

```
libmsquic
libtatsu
localai
markdown-oxide
mkdocs-material
node@22
onnx
pytest
python-freethreading
rip2
rust-script
sequoia-sq
surfer
tcl-tk@8
toml2json
typos-lsp
vipsdisp
wasi-libc
wasi-runtimes
wasm-component-ld
zizmor
==> New Casks
acronis-true-image-cleanup-tool
ba-connected
bobhelper
claude
cocoapacketanalyzer
default-handler
djuced
dockside
follow@alpha
font-eldur
font-greybeard
font-parkinsans
font-recursive-desktop
font-zpix
```

Summary

Jobs

test

Run details

Usage

Workflow file

test

succeeded 19 minutes ago in 36s

Search logs

Set up C++ environment 27s

```
github-copilot-for-xcode
huggingchat
hyperconnect
jet-pilot
langgraph-studio
lunatask
mailsteward
neohtop
sketchup
viz
whimsical
yellowdot
==> Outdated Formulae
aws-sam-cli
azure-cli
bazelisk
bicep
c-ares
cmake
curl
expect
gcc
gh
gnupg
harfbuzz
icu4c@75
libgpg-error
libnghttp2
libpq
node@20
openjdk
openssl@3
pixman
python-packaging
selenium-server
sqlite
```

Summary

Jobs

test

Run details

Usage

Workflow file

test

succeeded 19 minutes ago in 36s

Search logs

Set up C++ environment 27s

```
swiftformat
tcl-tk
unxip
wget
xcbeautify
xcodes
==> Outdated Casks
session-manager-plugin

You have 29 outdated formulae and 1 outdated cask installed.
You can upgrade them with brew upgrade
or list them with brew outdated.
gcc 14.2.0 is already installed but outdated (so it will be upgraded).
==> Downloading https://ghcr.io/v2/homebrew/core/gcc/manifests/14.2.0_1
==> Fetching gcc
==> Downloading https://ghcr.io/v2/homebrew/core/gcc/blobs/sha256:4d94c81d3aea48d77d6e7d7d359d05b48024bac44ade59e08dc3fe0f62539a17
==> Upgrading gcc
  14.2.0 -> 14.2.0_1
==> Pouring gcc--14.2.0_1.arm64_sonoma.bottle.tar.gz
🍺  /opt/homebrew/Cellar/gcc/14.2.0_1: 1,913 files, 459.5MB
```

Build the server 1s

```
▶ Run g++ -std=c++11 -o server server.cpp  # Компиляция серверной части с указанием стандарта
```

Run tests 0s

```
▶ Run g++ -std=c++11 -o run_tests test.cpp  # Компиляция тестов с указанием стандарта
test.cpp:39:27: warning: ISO C++11 does not allow conversion from string literal to 'char *' [-Wwritable-strings]
        char* argv1[] = { "program_name", "arg1", "arg2" }; // Неправильное количество аргументов
                        ^
test.cpp:39:43: warning: ISO C++11 does not allow conversion from string literal to 'char *' [-Wwritable-strings]
        char* argv1[] = { "program_name", "arg1", "arg2" }; // Неправильное количество аргументов
                                          ^
test.cpp:39:51: warning: ISO C++11 does not allow conversion from string literal to 'char *' [-Wwritable-strings]
        char* argv1[] = { "program_name", "arg1", "arg2" }; // Неправильное количество аргументов
```

Summary

Jobs

test

Run details

Usage

Workflow file

test

succeeded 19 minutes ago in 36s

Search logs

Run tests 0s

```
                                          ^
test.cpp:47:27: warning: ISO C++11 does not allow conversion from string literal to 'char *' [-Wwritable-strings]
        char* argv2[] = { "program_name", "-5" }; // Неправильное значение
                        ^
test.cpp:47:43: warning: ISO C++11 does not allow conversion from string literal to 'char *' [-Wwritable-strings]
        char* argv2[] = { "program_name", "-5" }; // Неправильное значение
                                          ^
test.cpp:55:27: warning: ISO C++11 does not allow conversion from string literal to 'char *' [-Wwritable-strings]
        char* argv3[] = { "program_name", "150" }; // Неправильное значение
                        ^
test.cpp:55:43: warning: ISO C++11 does not allow conversion from string literal to 'char *' [-Wwritable-strings]
        char* argv3[] = { "program_name", "150" }; // Неправильное значение
                                          ^
7 warnings generated.
Test passed: Error: Incorrect number of arguments. Expected 1 argument.
Test passed: Error: Argument out of range (0-100).
Test passed: Error: Argument out of range (0-100).
All tests passed!
```

Post Checkout code 0s

```
Post job cleanup.
/opt/homebrew/bin/git version
git version 2.47.0
Copying '/Users/runner/.gitconfig' to '/Users/runner/work/_temp/f4f1f288-727e-4ac0-8229-3c3f8c5dbc56/.gitconfig'
Temporarily overriding HOME='/Users/runner/work/_temp/f4f1f288-727e-4ac0-8229-3c3f8c5dbc56' before making global git config changes
Adding repository directory to the temporary git global config as a safe directory
/opt/homebrew/bin/git config --global --add safe.directory /Users/runner/work/os-task5-Oblachko-Js/os-task5-Oblachko-Js
/opt/homebrew/bin/git config --local --name-only --get-regexp core\.sshCommand
/opt/homebrew/bin/git submodule foreach --recursive sh -c "git config --local --name-only --get-regexp 'core\.sshCommand' && git config
--local --unset-all 'core.sshCommand' || :"
/opt/homebrew/bin/git config --local --name-only --get-regexp http\.https\:\/\/github\.com\/\.extraheader
http.https://github.com/.extraheader
/opt/homebrew/bin/git config --local --unset-all http.https://github.com/.extraheader
/opt/homebrew/bin/git submodule foreach --recursive sh -c "git config --local --name-only --get-regexp
'http\.https\:\/\/github\.com\/\.extraheader' && git config --local --unset-all 'http.https://github.com/.extraheader' || :"
```

Summary

Jobs

test

Run details

Usage

Workflow file

test

succeeded 20 minutes ago in 36s

Search logs

Run tests 0s

```
test.cpp:55:27: warning: ISO C++11 does not allow conversion from string literal to 'char *' [-Wwritable-strings]
        char* argv3[] = { "program_name", "150" }; // Неправильное значение
                        ^
test.cpp:55:43: warning: ISO C++11 does not allow conversion from string literal to 'char *' [-Wwritable-strings]
        char* argv3[] = { "program_name", "150" }; // Неправильное значение
                                          ^
7 warnings generated.
Test passed: Error: Incorrect number of arguments. Expected 1 argument.
Test passed: Error: Argument out of range (0-100).
Test passed: Error: Argument out of range (0-100).
All tests passed!
```

Post Checkout code 0s

```
Post job cleanup.
/opt/homebrew/bin/git version
git version 2.47.0
Copying '/Users/runner/.gitconfig' to '/Users/runner/work/_temp/f4f1f288-727e-4ac0-8229-3c3f8c5dbc56/.gitconfig'
Temporarily overriding HOME='/Users/runner/work/_temp/f4f1f288-727e-4ac0-8229-3c3f8c5dbc56' before making global git config changes
Adding repository directory to the temporary git global config as a safe directory
/opt/homebrew/bin/git config --global --add safe.directory /Users/runner/work/os-task5-Oblachko-Js/os-task5-Oblachko-Js
/opt/homebrew/bin/git config --local --name-only --get-regexp core\.sshCommand
/opt/homebrew/bin/git submodule foreach --recursive sh -c "git config --local --name-only --get-regexp 'core\.sshCommand' && git config
--local --unset-all 'core.sshCommand' || :"
/opt/homebrew/bin/git config --local --name-only --get-regexp http\.https\:\/\/github\.com\/\.extraheader
http.https://github.com/.extraheader
/opt/homebrew/bin/git config --local --unset-all http.https://github.com/.extraheader
/opt/homebrew/bin/git submodule foreach --recursive sh -c "git config --local --name-only --get-regexp
'http\.https\:\/\/github\.com\/\.extraheader' && git config --local --unset-all 'http.https://github.com/.extraheader' || :"
```

Complete job 1s

```
Cleaning up orphan processes
```

Исходный код тестов:

test.cpp

```
#include <iostream>

#include <cassert>

#include <stdexcept>

#include <string>


// Функция проверки на простоту

bool is_prime(int num) {

    if (num <= 1) return false;

    for (int i = 2; i <= num / 2; ++i) {

        if (num % i == 0) return false;

    }

    return true;

}


// Функция для проверки аргументов командной строки

void check_arguments(int argc, char* argv[]) {

    if (argc != 2) {

        throw std::invalid_argument("Error: Incorrect number of arguments. Expected
1 argument.");

    }


    int number = std::stoi(argv[1]);
```

```
    if (number < 0 || number > 100) {
        throw std::out_of_range("Error: Argument out of range (0-100).");
    }
}

// Основная функция для запуска тестов.
int main(int argc, char* argv[]) {
    // Тесты на простоту
    assert(is_prime(2) == true);
    assert(is_prime(3) == true);
    assert(is_prime(4) == false);
    assert(is_prime(5) == true);
    assert(is_prime(9) == false);
    assert(is_prime(11) == true);

    // Тесты для проверки аргументов
    try {
        char* argv1[] = { "program_name", "arg1", "arg2" }; // Неправильное
количество аргументов
        check_arguments(3, argv1);
        assert(false); // Если функция не выбросила исключение, тест провален
    } catch (const std::invalid_argument& e) {
        std::cout << "Test passed: " << e.what() << std::endl;
    }
```

```
    try {
        char* argv2[] = { "program_name", "-5" }; // Неправильное значение
        check_arguments(2, argv2);
        assert(false); // Если функция не выбросила исключение, тест провален
    } catch (const std::out_of_range& e) {
        std::cout << "Test passed: " << e.what() << std::endl;
    }

    try {
        char* argv3[] = { "program_name", "150" }; // Неправильное значение
        check_arguments(2, argv3);
        assert(false); // Если функция не выбросила исключение, тест провален
    } catch (const std::out_of_range& e) {
        std::cout << "Test passed: " << e.what() << std::endl;
    }

    std::cout << "All tests passed!" << std::endl;

    return 0;
}
```

Выводы:

В ходе работы над проектом были изучены принципы организации непрерывной интеграции приложений с использованием облачных технологий. Рассмотрены ключевые аспекты, такие как автоматизация сборки и тестирования кода, использование систем контроля версий и интеграция с облачными платформами.Изучены инструменты и технологии, позволяющие обеспечить эффективный процесс CI/CD, включая настройки конфигурационных файлов для автоматизации рабочих процессов. Обсуждены преимущества применения облачных решений для повышения доступности, масштабируемости и надежности разработки программного обеспечения.Также был проанализирован опыт использования GitHub Actions для реализации CI/CD в проектах на C++, что позволило получить практические навыки в настройке автоматизированных процессов и работе с логами сборки. В результате получены знания, которые помогут в дальнейшем оптимизировать процессы разработки и улучшить качество кода.